

Quality Assurance Plan

What "quality" means for this tool, where the risks sit, and how we will test it, from strategy down to specific test cases.

Scope · SDR live-call tool, CSV → SQLite ingest, traction log, manager view

Baseline · 23/23 tests pass · tsc clean · eslint clean

Date · Oct 1, 2026

1 Executive summary

The tool was built around three goals: **instant results** while the rep fills the intake rail, **a database as the source of truth** that unlocks a data loop, and **oversight for sales managers** across programs and reps. The code is in good shape: small, typed, with the core math and rule engine under test. The larger quality risks are not in the code. They sit in three places:

- **Meaning:** who confirms the rule interpretations are right?
- **Measurement:** a "send" is logged on a click, and outcomes are never recorded.
- **Adoption:** will reps trust the tool on a live call?

QUESTION 1

Is the answer right?

Would a rep be wrong to trust the ranking? Failure costs customer trust and could become a compliance problem.

QUESTION 2

Is the signal true?

Can managers decide things based on what the database records? Failure means confident decisions from bad data.

QUESTION 3

Does it hold up on a call?

Fast and smooth enough that reps keep using it? Failure means reps quietly stop using the tool.

Recommended QA priority

Correctness

→

Signal integrity

→

Speed

Speed is real and was prioritized well, but with 49 programs (~10 KB) running in the browser it is mostly solved. It is also the cheapest of the three to fix later.

2 Product goals → quality questions

GOAL	WHAT QUALITY MEANS HERE	HOW WE WILL KNOW
Instant results Ranking in useMemo , no network during a call	Speed means <i>time to a confident answer</i> , not milliseconds. The rep should know what to say next: open questions, "one answer away", talk tracks.	Under 16 ms per keystroke with CPU throttled 4×. Zero network requests while typing. Shadow real calls and time intake → first proposal.

Database as source of truth CSV hashed → snapshot → normalized tables	<i>Provenance</i> : we can always say which rate sheet produced a given recommendation, and a bad CSV can never quietly go live.	Snapshot lifecycle tests (new, repeat, revert, malformed). Every send traces to a <code>lender_program</code> row and its snapshot.
Manager oversight Traction log + <code>/manager</code>	The numbers mean what they appear to mean. Sends are never silently dropped or doubled, and the permissions hold.	Aggregates checked against hand-written SQL. A full E2E loop from SDR send to manager view. Tests of every role and session-expiry path.

3 Strategic risks and tensions

Measurement risks in the data loop

- **"Send" ≠ sent.** A row is logged when the rep clicks the mail button. The `mailto:` is only a draft, so we can't see whether it was delivered or edited.
- **No outcome.** We know what was recommended, not whether it funded. A loop with no result tells us what is *popular*, not what is *good*. This is the biggest gap for the data-loop story.
- **Goodhart risk.** Once managers watch send counts, reps will optimize for sends. Plan for it before it happens.
- **Perceived surveillance.** If SDRs experience the tool as monitoring, adoption drops. That is also a quality problem.

Tensions between the goals

TENSION	WHAT IT COSTS
Speed vs data loop	Matching runs in the browser with no network, so <i>nothing about the call is captured</i> : not the intake, not the tier, not where the program ranked. The loop only sees the final click.
Oversight vs rep trust	More visibility for managers can feel like being watched. Decide what reps can see about themselves.
Design mock vs CSV as truth	Ranking weights were tuned to reproduce the mock's top 3 for one sample customer. That fits one example, not what lenders actually approve.

4 Who decides what "correct" means

Testing needs a reference for the right answer. Today there are three, and none is final:

REFERENCE	STATUS	WHAT IT NEEDS
Rule semantics	Our own interpretation of CSV phrases. Example: "Requires business plan for startups" is a <i>watch item</i> , not a failure.	Sign-off from someone who knows lending (credit or partnerships).

Paper design mock	Known inconsistencies. The bucket counts 19/9/4/5 can't all be produced by any one precedence; the code produces 20/9/3/5.	Use it for layout and copy, not as the final word on logic.
Funding outcomes	Not captured.	A funded / declined field on each send, fed back into the database.

Until someone owns these decisions, tests only prove the code matches *our reading* of the rules, not that the rules are right.

5 QA strategy in layers

LAYER	QUESTION	HOW	STATUS
Verification	Did we build it right?	Unit tests on rules, money and ingest. Type checks, lint.	Mostly in place
Validation	Did we build the right thing?	Shadow 3–5 real SDR calls. A lending expert reviews the rule semantics.	Not started
Data integrity	Is the signal true?	Logging that can't silently drop or duplicate. Provenance from each send to its snapshot.	Gaps found
In production	Still true next month?	Ingest alerts. A warning when the rate sheet is stale. A check that send volume looks plausible.	Not started
Outcome	Does it help?	Feed funded / declined back into the database. Compare recommended with funded.	Not designed

6 Technical findings from code review

Found by reading the code. Each has high confidence but still needs to be reproduced before it's treated as a confirmed bug.

	SEVERITY	PILLAR	FINDING
1	High	Data	Reverting the CSV doesn't revert the app. An older CSV's hash already exists, so ingest returns <code>exists</code> and creates no new snapshot. <code>latestSnapshot</code> picks the highest id, so the app keeps serving the newer sheet. <code>src/db/ingest.ts:46</code> · <code>src/db/queries.ts:7</code>
2	High	Data loop	The send log can fail silently. It's a fire-and-forget <code>fetch</code> that never checks the response. With an expired session (401), a 400 or no network, the rep still sees "Draft copied" and the row is lost. A double-click logs two rows. <code>src/components/MailLink.tsx:24</code>
3	Medium	Rules	Negative cash flow can't be entered. The input strips <code>-</code> . The <code>positive_cash_flow</code> failure and an infinite debt ratio can only happen at exactly \$0. <code>src/components/intake/IntakeRail.tsx:13</code> , <code>:291</code>
4	Medium	Data	Ingest has no validation. A non-numeric cell becomes <code>NaN</code> , hits NOT NULL and crashes boot. The collateral enum is only checked in TypeScript. A new industry spelling never matches, because <code>INDUSTRIES</code> is hardcoded. <code>src/db/ingest.ts:36</code> · <code>src/lib/intake.ts:96</code>
5	Medium	Security	<code>/api/programs</code> needs no login and nothing calls it. It exposes rep-only thresholds (minimum credit, debt caps) to anyone. <code>src/app/api/programs/route.ts</code>
6	Low	Oversight	Managers can use the SDR screen and log sends. Their sends appear in "most recommended by", but the rep filter only lists SDRs. <code>src/app/page.tsx:14</code> · <code>src/db/traction.ts:114</code>
7	Low	Oversight	"Programs with no sends" can go negative. <code>programsWithSends</code> counts across all snapshots, while <code>programsTotal</code> counts only the latest. <code>src/db/traction.ts:113</code>
8	Low	UX / speed	The cursor may jump while typing money. The money input reformats on every keystroke, so editing the middle of a number may move the cursor. <code>src/components/intake/IntakeRail.tsx:31</code>
9	Low	Data loop	Long comparison emails may be cut off. A 4-program <code>mailto:</code> may exceed some mail clients' URL limits (~2k characters). The clipboard copy is the fallback. <code>src/lib/copy.ts:78</code>
10	Low	Docs	<code>context.md</code> is out of date. It says intake persists in <code>localStorage</code> (the code keeps it in memory, so the seed snippet does nothing). It contradicts itself on which migration seeds passwords. It describes 72 demo sends; the database has 1.

7 Test plan by area

7.1 Speed Goal 1

- **Budget:** time `evaluate` + `rank` with `performance.now`. Target under 16 ms per keystroke with CPU throttled 4× and 6×.
- **Headroom:** synthetic CSVs with 500 and 5,000 rows. Every program is recomputed and the whole list redraws on each keystroke, so find where it starts to lag.
- **No network:** watch the network panel while typing, pasting, clicking Clear and switching presets. Expect zero requests.
- **Offline mid-call:** ranking keeps working; finding #2 should show up.
- **First load:** time to first meaningful paint on `/` with the server-rendered program list.

7.2 Database as source of truth Goal 2

- **Lifecycle:** start with `--reset`, then restart (no duplicate snapshot). A new CSV becomes snapshot 2 and is served. **Revert the CSV (finding #1).** Run `PRAGMA foreign_key_check`.
- **Malformed CSVs:** unknown phrase (should fail loudly), non-numeric value, BOM, CRLF line endings, quoted commas, missing or extra column, duplicate lender + program, new industry string, lowercase collateral.
- **Migrations:** replay on an empty database; the schema matches `drizzle/meta`.
- **POST `/api/recommendations`** (call the route handlers directly in Vitest; no new dependencies):
 - 401 without a session
 - 400 for: a proposal with 2 programs, a comparison with 1, a non-integer id, an unknown id, a blank email
 - duplicate ids collapse
 - a blank `customerName` is currently accepted (decide whether that's OK)
- **Across snapshots:** sends from snapshot 1 and snapshot 2 merge under one lender + program-type key.

7.3 Manager oversight Goal 3

- **Access rules:**
 - signed out → `/login` from both pages
 - an SDR on `/manager` is sent to `/`, and `/api/traction` returns 403
 - a manager gets 200 from `/api/traction`
- **Correct numbers:** on a fixed seed, check against hand-written SQL:
 - totals, comparisons vs solo sends, ranking order
 - pair counts are symmetric ($A-B = B-A$)
 - top 3 reps and the "N other reps" line, last sent
- **Filters:**
 - a send at 11:30 pm Central on the last day of the month counts in that month
 - November (daylight-saving change)
 - "All time"; an upcoming month shows the empty state
 - the rep filter, including how managers' own sends appear (finding #6)

- "Show all" past 8 programs
- **End to end:** in two browser sessions, an SDR sends one proposal and one 3-program comparison, then the manager refreshes and sees exactly those. Repeat with the SDR's session expired.

7.4 Rule engine (what every goal depends on)

Boundary tests

- amount: at min, at max, min-1, max+1
- credit: band exactly at the floor
- time in business: 0 / 5 / 6 / 11 / 12 months, 1 year
- debt ratio exactly at the cap
- turnaround exactly at the target
- injection exactly 10%; ownership exactly 20%
- cash flow of \$0

Changes and invariants

- Real estate purchase shows owner-occupancy and updates the section counter; Refinance fails `no_refi`.
- Answering an open question moves potential to strong or ineligible, never strong to potential.
- Better credit, more years or a lower ratio never lowers a tier.
- Pin the default Rosa Tacos call next to the existing Marisol fixture.
- Compare: limit of 4; an ineligible program drops out; swapping and removing keep the order; one column left logs as a proposal.

7.5 Cross-cutting

- Mail button in Chrome with no mail client; clipboard in Safari and Firefox.
- Phone, tablet and the `lg` breakpoint where the intake rail moves.
- Keyboard use for the segmented buttons and the ownership stepper.
- Visual comparison against the Paper artboards.

8 Execution plan

- 1 **Reproduce findings 1–4** (~30 min, command line and browser).
- 2 **Rule boundary and invariant tests** in Vitest.
- 3 **Ingest robustness tests** with malformed CSV fixtures.
- 4 **API contract tests** calling route handlers directly.
- 5 **SDR → manager end-to-end loop** in two browser sessions.
- 6 **Performance profiling** and stress CSVs.
- 7 **Responsive and visual pass** against Paper.

Exit criteria for a pilot

- Findings 1 and 2 fixed; 3–5 fixed or accepted with a written reason.
- Rule semantics signed off by a lending owner.
- Traction numbers match SQL on a seeded fixture.
- Under 16 ms per keystroke at 4× throttle.
- One full SDR → manager loop verified, including an expired session.

After launch

- Alert when ingest fails, and warn when the rate sheet is stale.
- Watch for anomalies in send volume per rep.
- Add funded / declined so the loop measures outcomes, not activity.

Baseline at time of writing: Vitest 5 files / 23 tests passing; `tsc --noEmit` and eslint clean. Database: 1 snapshot (49 programs), 10 users (8 SDRs, 2 managers), 1 logged send.